Step 1: Understand the assignment

Your program needs to:

1. Create a family tree with 3 generations.
2. Give each person two blood type alleles (A, B, or O).
3. Give each child one allele from each parent.
4. Use `malloc` to create people in memory.
5. Use `free` to release the memory when finished.
6. Pass the `check50` tests and show no memory leaks in Valgrind.

Step 2: Write your pseudocode

Pseudocode explains what your program will do using simple English instead of C code.

Pseudocode for `create_family`

1. Create memory for one new person.

2. If generations is greater than 1:

3. Create two parents using recursion.

4. Give the child one random allele from each parent.

5. Otherwise (base case):

6. Set both parents to NULL.

7. Give the person two random alleles.

8. Return the new person.

Base case: When `generations == 1`, the person has no parents.

Pseudocode for `free_family`

1. If the person is NULL, stop. (Base case)

2. Find the person's first parent.

3. Free the first parent's family recursively.

4. Find the person's second parent.

5. Free the second parent's family recursively.

6. Free the current person.

1. If the person is NULL, stop. (Base case)

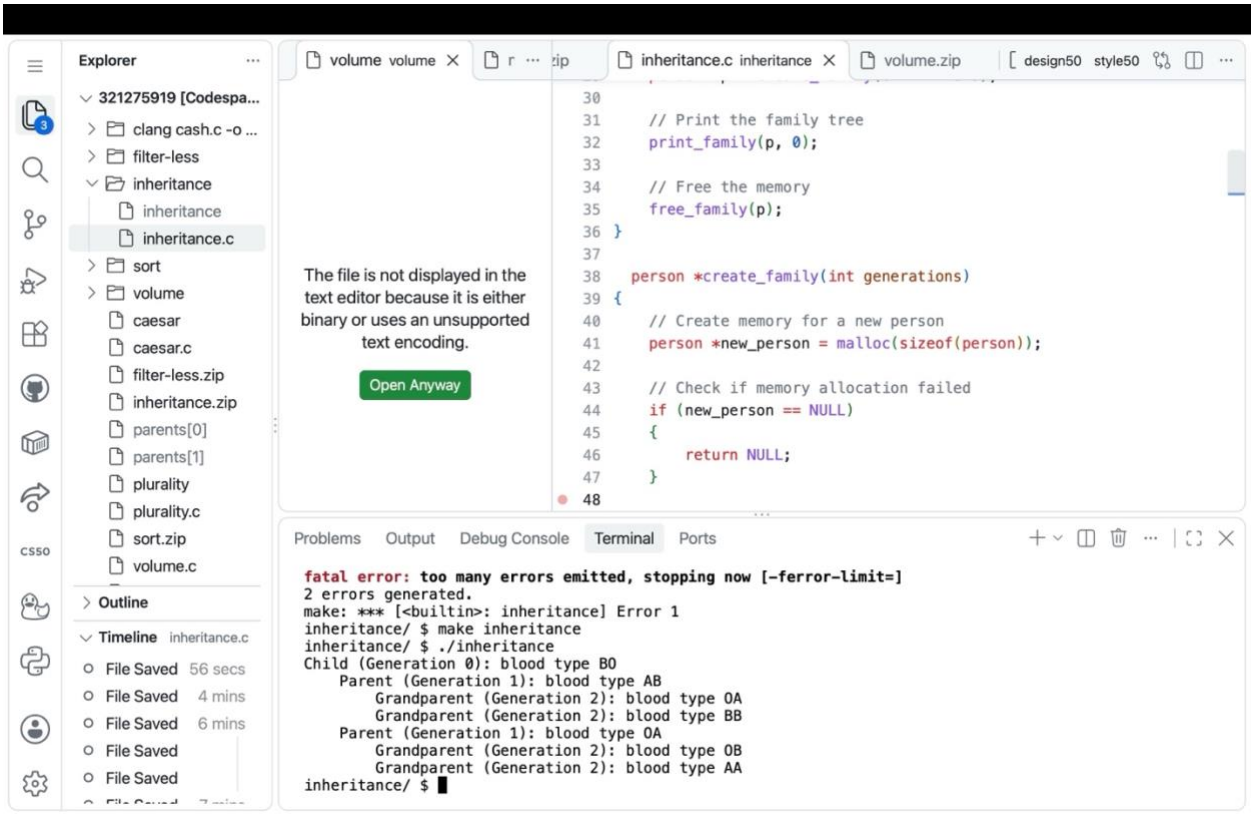
2. Find the person's first parent.

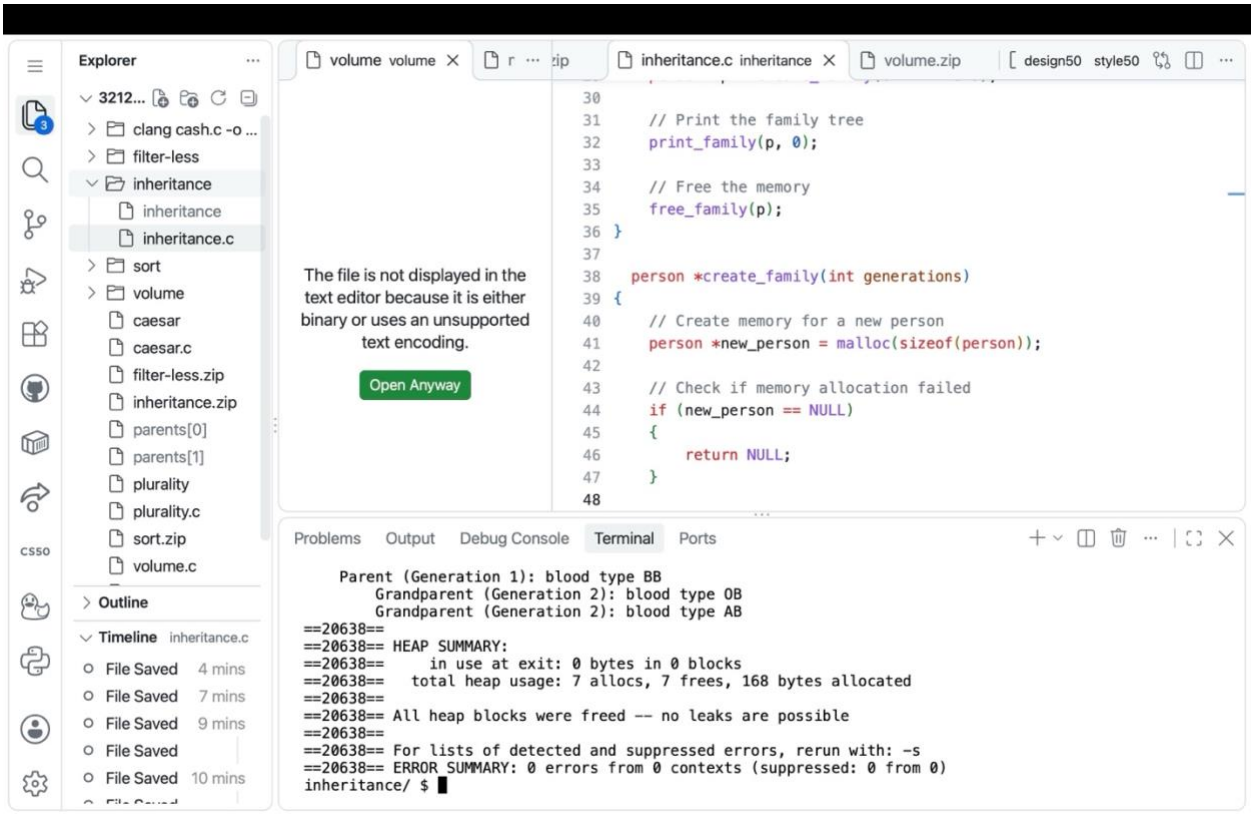
3. Free the first parent's family recursively.

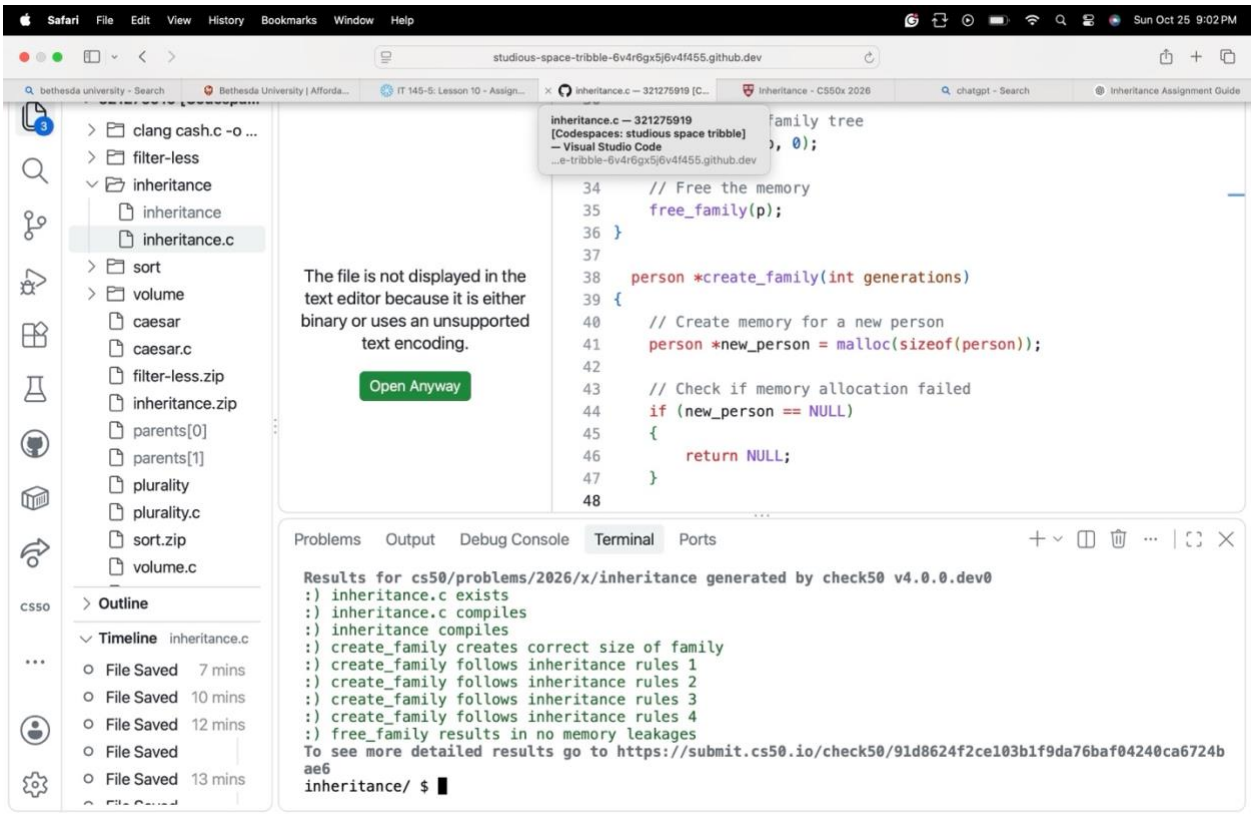
4. Find the person's second parent.

5. Free the second parent's family recursively.

6. Free the current person.




Explorer
volume volume
inheritance.c inheritance
volume.zip
design50 style50
clang cash.c -o ...
filter-less
inheritance
inheritance
inheritance.c
sort
volume
caesar
caesar.c
filter-less.zip
inheritance.zip
parents[0]
parents[1]
plurality
plurality.c
sort.zip
volume.c
Outline
Timeline inheritance.c
File Saved 4 mins
File Saved 7 mins
File Saved 9 mins
File Saved
File Saved 10 mins
The file is not displayed in the text editor because it is either binary or uses an unsupported text encoding.
Open Anyway
30
31 // Print the family tree
32 print_family(p, 0);
33
34 // Free the memory
35 free_family(p);
36 }
37
38 person *create_family(int generations)
39 {
40 // Create memory for a new person
41 person *new_person = malloc(sizeof(person));
42
43 // Check if memory allocation failed
44 if (new_person == NULL)
45 {
46 return NULL;
47 }
48
Problems Output Debug Console Terminal Ports
Parent (Generation 1): blood type BB
Grandparent (Generation 2): blood type OB
Grandparent (Generation 2): blood type AB
==20638==
==20638== HEAP SUMMARY:
==20638== in use at exit: 0 bytes in 0 blocks
==20638== total heap usage: 7 allocs, 7 frees, 168 bytes allocated
==20638==
==20638== All heap blocks were freed -- no leaks are possible
==20638==
==20638== For lists of detected and suppressed errors, rerun with: -s
==20638== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
inheritance/ $


Safari File Edit View History Bookmarks Window Help
Sun Oct 25 9:02 PM
studious-space-tribble-6v4r6gx5j6v4f455.github.dev
inheritance.c — 321275919 [Codespaces: studious space tribble] — Visual Studio Code
...e-tribble-6v4r6gx5j6v4f455.github.dev
Explorer
clang cash.c -o ...
filter-less
inheritance
inheritance
inheritance.c
sort
volume
caesar
caesar.c
filter-less.zip
inheritance.zip
parents[0]
parents[1]
plurality
plurality.c
sort.zip
volume.c
Outline
Timeline inheritance.c
File Saved 7 mins
File Saved 10 mins
File Saved 12 mins
File Saved
File Saved 13 mins
The file is not displayed in the text editor because it is either binary or uses an unsupported text encoding.
Open Anyway
34 // Free the memory
35 free_family(p);
36 }
37
38 person *create_family(int generations)
39 {
40 // Create memory for a new person
41 person *new_person = malloc(sizeof(person));
42
43 // Check if memory allocation failed
44 if (new_person == NULL)
45 {
46 return NULL;
47 }
48
Problems Output Debug Console Terminal Ports
Results for cs50/problems/2026/x/inheritance generated by check50 v4.0.0.dev0
:) inheritance.c exists
:) inheritance.c compiles
:) inheritance compiles
:) create_family creates correct size of family
:) create_family follows inheritance rules 1
:) create_family follows inheritance rules 2
:) create_family follows inheritance rules 3
:) create_family follows inheritance rules 4
:) free_family results in no memory leakages
To see more detailed results go to https://submit.cs50.io/check50/91d8624f2ce103b1f9da76baf04240ca6724bae6
inheritance/ $

We need to free the parents first because the child has pointers that connect to them. If we free

Thechild first, we can no longer safely access those pointers. Freeing the parents before the child makes sure all the memory is properly released and prevents memory errors.